Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: April 22, 2024,
Submission Date: April 29, 2024

Lab Report 1, 2 & 3: Using Assembly Language to Program 8086 Microprocessor to do Different Tasks.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Conversion From Lowercase to Uppercase.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
DATA    SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'ENTER A LOWERCASE LETTER: $'
    MSG2 DB CR,LF, 'IN UPPER CASE IT IS: '
    CHAR DB ?, '$'
DATA    ENDS
CODE    SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 9
    INT 21h
    MOV AH, 1
```

```
19      INT 21h
20
21      SUB AL, 20H
22      MOV CHAR, AL
23
24      LEA DX,MSG2
25      MOV AH, 9
26      INT 21h
27
28      MOV AH, 4CH
29      INT 21h
30
31  CODE    ENDS
32
33  END START
```

Output

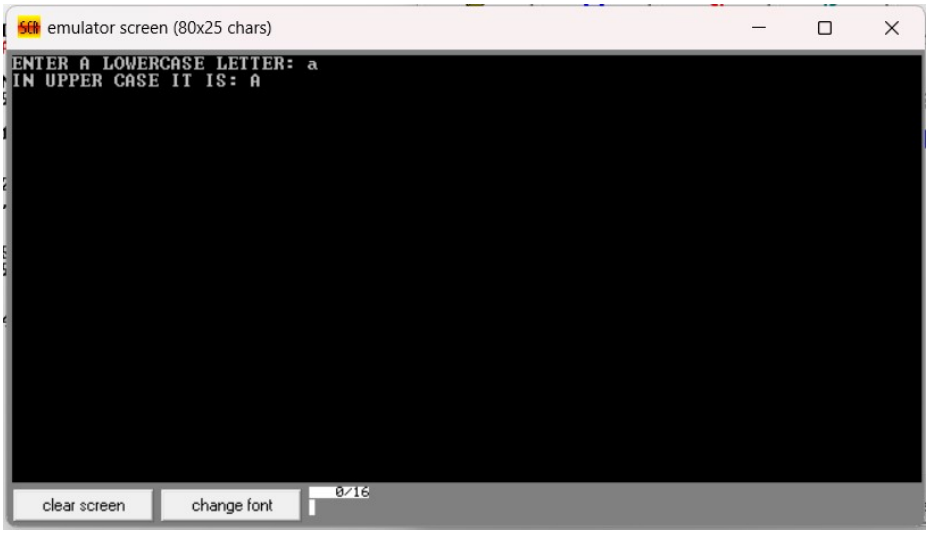


Figure 1: Lowercase to Uppercase

Conversion From Uppercase to Lowercase.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
    DATA    SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB 'ENTER A UPPERCASE LETTER: $'
    MSG2 DB CR,LF, 'IN LOWERCASE IT IS: '
    CHAR DB ?, '$'
DATA    ENDS
CODE    SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 9
    INT 21h
    MOV AH, 1
```

```
19      INT 21h
20
21      ADD AL, 20H
22      MOV CHAR, AL
23
24      LEA DX,MSG2
25      MOV AH, 9
26      INT 21h
27
28      MOV AH, 4CH
29      INT 21h
30
31  CODE    ENDS
32
33  END START
```

Output

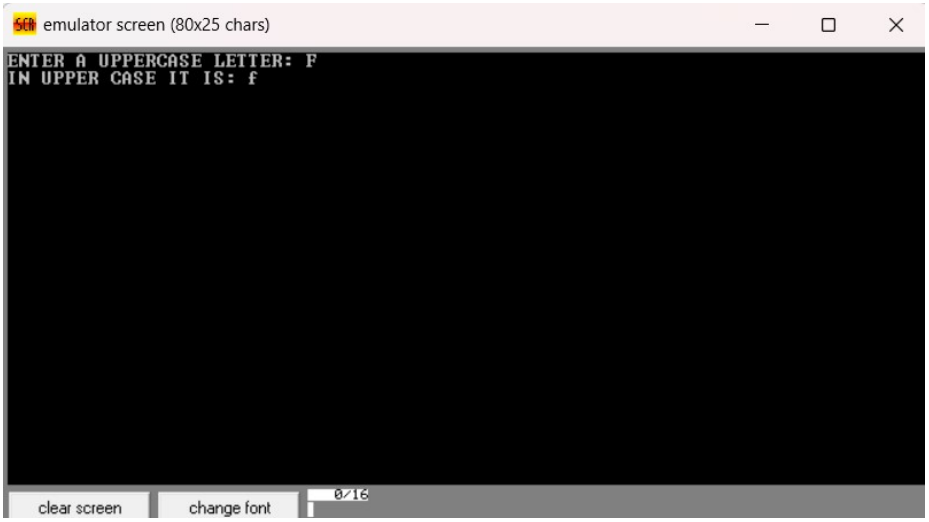


Figure 2: Uppercase to Lowercase

Adding Two Numbers

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
DATA    SEGMENT
    CR EQU 0DH
    LF EQU 0AH

    MSG1 DB 'ENTER FIRST NUMBER: $'
    NUM1 DB ?
    MSG2 DB CR,LF,'ENTER SECOND NUMBER: $'
    NUM2 DB ?
    MSG3 DB CR, LF, 'SUM = '
    NUM3 DW ?,'$'
DATA    ENDS

CODE    SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA
    MOV DS, AX
```

```
19
20      LEA DX, MSG1
21      MOV AH, 9
22      INT 21h
23
24      MOV AH, 1
25      INT 21h
26      SUB AL, '0'
27      MOV NUM1, AL
28
29      LEA DX, MSG2
30      MOV AH, 9
31      INT 21h
32
33      MOV AH, 1
34      INT 21h
35      SUB AL, '0'
36      MOV NUM2, AL
37
38      MOV BH,0
39      MOV BL, NUM1
40      MOV AH, 0
41      MOV AL, NUM2
42      ADD AX,BX
43      MOV NUM3, AX
44
45      ADD NUM3, '0'
46
47      LEA DX, MSG3
48      MOV AH, 9
49      INT 21h
50
51      MOV AH, 4CH
52      INT 21h
53
54  CODE    ENDS
55
56  END START
```

Output



Figure 3: Uppercase to Lowercase

Keywords

CR, LF Carriage Return & Line Feed. Both together is used to go to the next line. CR is used to go to next line and LF to go to the start of the line.

DB Data Byte. Used to declare type of variable.

MOV Move. Used to move data from the second register to the first.

LEA Load Effective Address. Used to finding address of variable, and addition or subtraction without affecting flags.

INT 21h Interrupt. Used to give emu8086 take back control.

SUB Subtract. Used to subtract the second value from the first one and store in the first one.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: April 29, 2024,
Submission Date: May 11, 2024

Lab Report 4: Using Assembly Language to Program 8086 Microprocessor to Solve Problems.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Problem 4.1

Problem Description

Write a code to find the min/max number from three numbers (Using registers).

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
—

DATA SEGMENT
    MSG1 DB "The max number is: "
    ANS DB ?, '$'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS DATA

START:
    MOV AX, DATA
    MOV DS, AX

    MOV AX, '8'
    MOV BX, '3'
    MOV CX, '6'

    CMP AX, BX
    JG CHK1

    MOV DX, BX
    JMP NXT1

CHK1:
    MOV DX, AX
```

```
25
26  NXT1:
27      CMP DX, CX
28      JG CHK2
29      MOV AX, CX
30      JMP NXT2
31
32  CHK2:
33      MOV AX, DX
34
35  NXT2:
36      MOV ANS, AL
37      LEA DX, MSG1
38      MOV AH, 9
39      INT 21H
40      MOV AH, 4CH
41      INT 21H
42
43  CODE ENDS
44  END START
```

Output

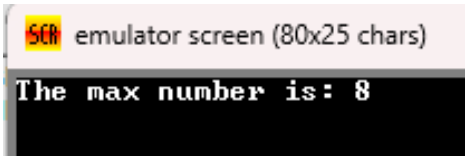


Figure 1: Finding min/max number

Problem 4.2

Problem Description

Write a program to display a "?", read two capital letters, and display them on the next line in alphabetical order.

Process

Code:

```
—

DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB CR, LF, '? $'
    CHAR1 DB ?
    CHAR2 DB ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    MOV CHAR1, AL

    MOV AH, 1
    INT 21H
    MOV CHAR2, AL

    MOV AL, CHAR1
    MOV BL, CHAR2

    CMP AL, BL
    JLE SKIP_SWAP
    MOV CHAR1, BL
    MOV CHAR2, AL

SKIP_SWAP:
    ; newline
    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
```

```
38      MOV DL, 0Ah
39      INT 21h
40
41      ; first character
42      MOV AH, 2
43      MOV DL, CHAR1
44      INT 21h
45
46      ; second character
47      MOV AH, 2
48      MOV DL, CHAR2
49      INT 21h
50
51      ; Terminate program
52      MOV AH, 4Ch
53      INT 21h
54
55  CODE ENDS
56
57  END START
```

Output

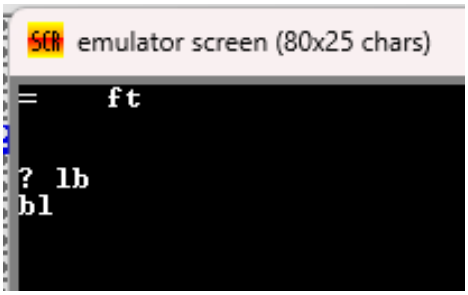


Figure 2: Ordering letters alphabetically

Problem 4.3

Problem Description

Write a program that will prompt the user to enter a HEX digit character(”0”,.....”9” or, ”A”,.....”F”), display it on the next line in decimal, and ask the user if he or she wants to do it again. If the user types ”Y” or ”y”, the program terminates. If the user enters an illegal character. prompt the user try again.

Process

Code:

```
—

DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB CR, LF, "Enter A HEX Digit: $"
    MSG2 DB CR, LF, "Do it again? $"
    MSG3 DB CR, LF, "In Decimal: $"
    MSG4 DB CR, LF, "In Decimal: 1$"
    MSG5 DB CR, LF, "Try Again.$"
    CHAR DB ?, '$'
    ANS DB ?, '$'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS: DATA

START:
    MOV AX, DATA
    MOV DS, AX

LOOP_START:
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H

    CMP AL, '0'
    JB NOT_NUM
```

```
30
31      CMP AL, '9'
32      JA NOT_NUM
33
34      MOV ANS, AL
35
36      LEA DX, MSG3
37      MOV AH, 9
38      INT 21H
39
40      MOV DL, ANS
41      MOV AH, 2
42      INT 21H
43
44      JMP LOOP_CHECK
45
46  NOT_NUM:
47      CMP AL, 'A'
48      JB ILLEGAL
49
50      CMP AL, 'F'
51      JA ILLEGAL
52
53      MOV ANS, AL
54      SUB ANS, 11H
55
56      LEA DX, MSG4
57      MOV AH, 9
58      INT 21H
59
60      MOV DL, ANS
61
62      MOV AH, 2
63      INT 21H
64      JMP LOOP_CHECK
65
66  ILLEGAL:
67      LEA DX, MSG5
68      MOV AH, 9
69      INT 21H
70
71  LOOP_CHECK:
72      LEA DX, MSG2
73      MOV AH, 9
74      INT 21H
75
76      MOV AH, 1
77      INT 21H
78
79      CMP AL, 'y'
80      JZ LOOP_START
81
82      CMP AL, 'Y'
83      JZ LOOP_START
84
85      MOV AH, 4CH
86      INT 21H
87
88  CODE ENDS
89  END START
```

Output

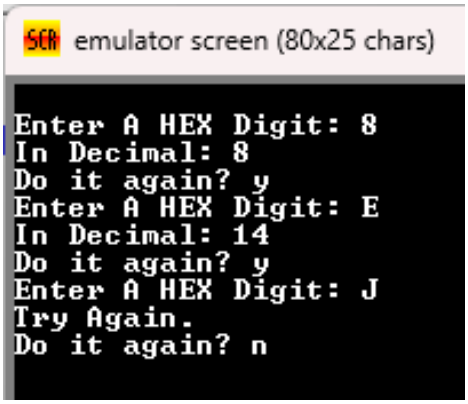


Figure 3: Converting HEX to Decimal

Problem 4.4

Problem Description

Write a program to read one of the HEX digits A-F, and display it on the next line in decimal.

Process

Code:

```
—

DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    MSG1 DB CR, LF, "Enter the HEX number : $"
    MSG2 DB CR, LF, "In decimal: 1$"
    NUM DB ?
    DEC_NUM DB ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS DATA

START:
    MOV AX, DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H
    MOV NUM, AL

    SUB NUM, 11H

    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    MOV DL, NUM

    MOV AH, 02H
    INT 21H
```

```
36      MOV AH, 4CH
37      INT 21H
38  CODE ENDS
39  END START
```

Output

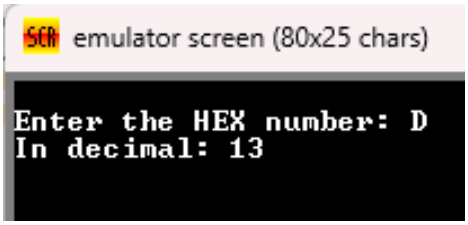


Figure 4: Converting HEX (A-F) to Decimal

Problem 4.5

Problem Description

Process

Code:

```
;
DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    STARS DB CR, LF, "**********$"
    COUNTER DW 10
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS DATA

START:
    MOV AX, DATA
    MOV DS, AX
    MOV CX, COUNTER

LOOP_START:
    LEA DX, STARS
    MOV AH, 9
    INT 21H

    DEC CX
    CMP CX, 0

    JE LOOP_END
    JMP LOOP_START

LOOP_END:
    MOV AH, 4CH
    INT 21H

CODE ENDS
END START
```

Output

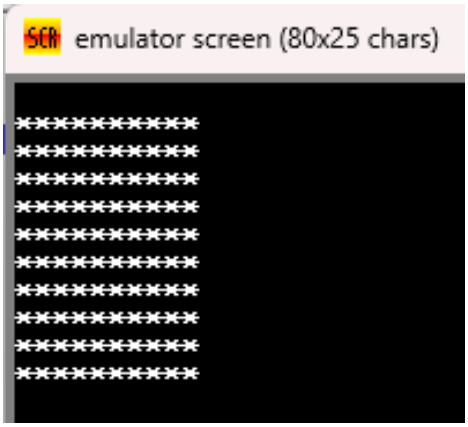


Figure 5: Printing a pattern using Loop

Keywords

These are the new keywords learned while solving the Problems

CMP Used to compare 2 provided byte/word.

JMP Used to jump to the provided address to proceed to the next instruction.

JA Used to jump if above/not below/equal instruction satisfies.

JB Used to jump if below/equal/ not above instruction satisfies.

JG Used to jump if greater/not less than/equal instruction satisfies.

JLE Used to jump if less than/equal/if not greater than instruction satisfies.

JZ Used to jump if equal/zero flag ZF = 1

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: May 11, 2024,
Submission Date: May 25, 2024

Lab Report 5: Using Assembly Language to Program 8086 Microprocessor to Solve Problems.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Problem 4.1

Problem Description

Write a code to take input from user as Binary and output it as Binary.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
    —

DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    CNT DB 17
DATA ENDS

CODE SEGMENT
    ASSUME CS: CODE, DS: DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR BX, BX
    MOV CX, 16

    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE EXIT

    AND AL, 0FH
```

```
25      SHL BX, 1
26      OR BL, AL
27
28      INT 21H
29
30      JMP INPUT
31
32  EXIT:
33      MOV DL, CR
34      MOV AH, 2
35      INT 21H
36      MOV DL, LF
37      INT 21H
38
39  PRINT:
40      DEC CNT
41      JZ END_
42
43      RCL BX, 1
44      JC PRINT1
45      JNC PRINT0
```

```
46
47  PRINT1:
48      MOV DL, '1'
49      MOV AH, 2
50      INT 21H
51
52      JMP PRINT
53
54  PRINT0:
55      MOV DL, '0'
56      MOV AH, 2
57      INT 21H
58
59      JMP PRINT
60
61  END_:
62      MOV AH, 4CH
63      INT 21H
64
65  CODE ENDS
66  END START
```

Output

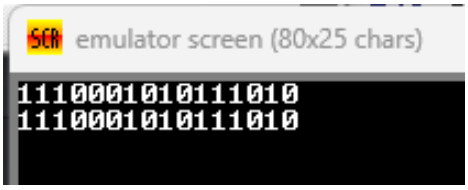


Figure 1: Binary input & output

Problem 4.2

Problem Description

Write a code to take input from user as Hexadecimal and output it as Hexadecimal.

Process

Code:

```
;
DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    CNT DB 4
DATA ENDS

CODE SEGMENT
    ASSUME CS: CODE, DS: DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR BX, BX
    MOV CX, 4

    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE EXIT

    CMP AL, 39H
    JG LETTER

    AND AL, 0FH
    JMP SHIFT

LETTER:
    SUB AL, 37H

SHIFT:
    SHL BX, CL
    OR BL, AL

    INT 21H

    JMP INPUT
```

```
40
41  EXIT:
42      MOV DL, CR
43      MOV AH, 2
44      INT 21H
45      MOV DL, LF
46      INT 21H
47
48  PRINT:
49      MOV DL, BH
50      SHR DL, 4
51      ROL BX, 4
52
53      CMP DL, 10
54      JL CONV_NUM
55      JGE CONV_CHAR
56
57  CONV_NUM:
58      OR DL, 30H
59      MOV AH, 2
60      INT 21H
61
62      LOOP PRINT
63
64  CONV_CHAR:
65      SUB DL, 9
66      OR DL, 40H
67      MOV AH, 2
68      INT 21H
69
70      LOOP PRINT
71
72  END1:
73      MOV AH, 4CH
74      INT 21H
75
76  CODE ENDS
77  ENDS START
```

Output

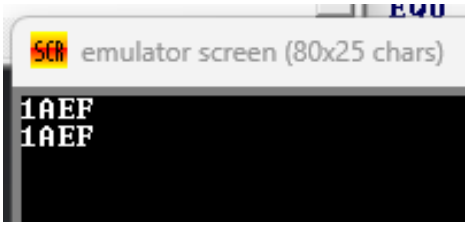


Figure 2: Hexadecimal input & output

Keywords

These are the new keywords learned while solving the Problems

EQU Used to declare a variable as a constant.

XOR Used to perform Exclusive-OR operation over each bit in a byte/word with the corresponding bit in another byte/word. Used here on the same variable to clear the register.

AND Used for adding each bit in a byte/word with the corresponding bit in another byte/word.

OR Used to multiply each bit in a byte/word with the corresponding bit in another byte/word.

JE Used to jump if equal/zero flag ZF = 1

SHL Used to shift bits of a byte/word towards left and put zero(S) in LSBs.

RCL Used to rotate bits of byte/word towards the left, i.e. MSB to CF and CF to LSB.

JC Used to jump if carry flag CF = 1

JNC Used to jump if no carry flag (CF = 0)

SHR Used to shift bits of a byte/word towards the right and put zero(S) in MSBs.

ROL Used to rotate bits of byte/word towards the left, i.e. MSB to LSB and to Carry Flag [CF].

JL Used to jump if less than/not greater than/equal instruction satisfies.

JGE Used to jump if greater than/equal/not less than instruction satisfies.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: May 25, 2024,
Submission Date: June 3, 2024

Lab Report 6: Using Assembly Language to Program 8086 Microprocessor to Solve Problems Related To Shifting & Stack.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Problem 6.1

Problem Description

Write a code to take two input from user as Binary and multiply them using shifting operation and output the result in binary.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
;
DATA SEGMENT
    INA DW ?
    INB DW ?
    COUNT DB 16
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE,DS:DATA

MAIN PROC
    MOV AX,DATA
    MOV DS,AX

    CALL MULTIPLY
    CALL BIN_OUT

    MOV AH, 4CH
    INT 21H
MAIN ENDP

MULTIPLY PROC
    CALL BIN_IN
```

```
    MOV INA, CX
    CALL BIN_IN
    MOV INB, CX

    MOV AX, INA
    MOV BX, INB
    PUSH AX
    PUSH BX
    XOR DX, DX

    LOOP_:
        TEST BX, 1
        JZ END_IF

        ADD DX, AX

    END_IF:
        SHL AX, 1
        SHR BX, 1

        JNZ LOOP_
        POP AX
        POP BX

        RET
MULTIPLY ENDP

BIN_IN PROC
    XOR BX, BX
    MOV AH, 1
    INT 21H

    INPUT:
        CMP AL, 0DH
        JE EXIT_

        AND AL, 0FH
        SHL BX, 1
        OR BL, AL

        INT 21H
        JMP INPUT

    EXIT_:
        MOV DL, 0DH
        MOV AH, 2
        INT 21H
        MOV DL, 0AH
        INT 21H

        MOV CX, BX
        RET
BIN_IN ENDP

BIN_OUT PROC
    MOV BX, DX

    PRINT:
            RCL BX, 1
            JC PRINT1
            JNC PRINT0

    PRINT1:
        CMP COUNT, 0
        JE END_

        MOV DL, '1'
        MOV AH, 2
        INT 21H
        DEC COUNT
        JMP PRINT

    PRINT0:
        CMP COUNT, 0
        JE END_

        MOV DL, '0'
        MOV AH, 2
        INT 21H
        DEC COUNT
        JMP PRINT

    END_:
        RET
```

```
107
108 BIN_OUT ENDP
109
110 END MAIN
```

Output

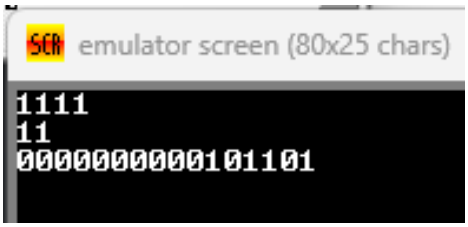


Figure 1: Binary input & output

Problem 6.2

Problem Description

Write a code to take input from user as string and then out the string in reverse. Use stack.

Process

Code:

```
—

DATA SEGMENT

DATA ENDS
CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR CX, CX
    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE EXIT_

    PUSH AX
    INC CX

    INT 21H
    JMP INPUT

EXIT_:
    MOV AH, 2
    MOV DL, 0DH
    INT 21H
    MOV DL, 0AH
    INT 21H

OUTPUT:
    POP DX
    INT 21H
    LOOP OUTPUT

CODE ENDS
END START
```

Output

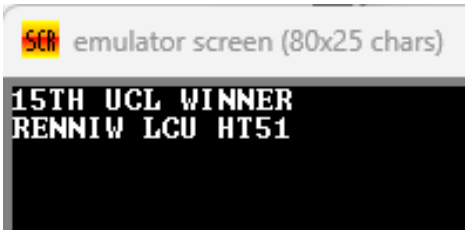


Figure 2: Hexadecimal input & output

Problem 6.2

Problem Description

Write a program that lets the user type some text, consisting of words separated by blanks, ending with a carriage return, and displays the text in same word order as entered, "this is a test" becomes "siht si a tset".

Process

Code:

```
—

DATA SEGMENT
    COUNT DW 0
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR CX,CX

    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE END_

    PUSH AX
    INC CX
    INT 21H
    JMP INPUT

END_:
    MOV DX, 0DH
    MOV AH, 2
    INT 21H
    MOV DX, 0AH
    INT 21H

    MOV BX, SP
    SUB BX, CX
    SUB BX, CX
```

```
36
37      XCHG BX, SP
38      PUSH 20H
39      XCHG BX, SP
40      INC COUNT
41
42  TO_NEW_STK:
43      POP DX
44      XCHG BX, SP
45      PUSH DX
46      XCHG BX, SP
47      INC COUNT
48
49      LOOP TO_NEW_STK
50
51      MOV CX, COUNT
52      MOV COUNT, 0
53      PUSH 20H
54      INC COUNT
55
56  REVERSE:
57      XCHG BX, SP
58      POP DX
59
60      XCHG BX, SP
61      CMP DL, 20H
62
63      JNE SKIP
64
65      MOV AH, 2
66
67  REV_PRINT:
68      POP DX
69      INT 21H
70
71      DEC COUNT
72      JNZ REV_PRINT
73
74      MOV DX, 20H
75
76  SKIP:
77      PUSH DX
78      INC COUNT
79
80      LOOP REVERSE
81
82      MOV AH, 4CH
83      INT 21H
84
85  CODE ENDS
86  END START
```

Output

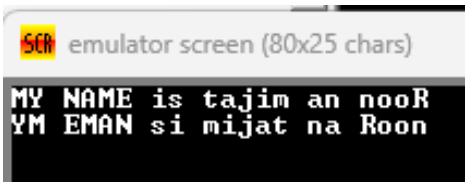


Figure 3: Hexadecimal input & output

Keywords

These are the new keywords learned while solving the Problems

PUSH Used to put a word at the top of the stack.

POP Used to get a word from the top of the stack to the provided location.

XCHG Used to exchange the data from two locations.

CALL Used to call a procedure and save their return address to the stack.

RET Used to return from the procedure to the main program.

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: June 3, 2024,
Submission Date: June 24, 2024

Lab Report 7: Using Assembly Language to Program 8086 Microprocessor to Solve Problems Related To Array.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Problem 7.1

Problem Description

Reverse an array.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
;-------------------------------------------------------------------------

DATA SEGMENT
    ARR DB 1, 2, 3, 4, 5, 6, 7, 8
DATA ENDS

CODE SEGMENT
    ASSUME DS:DATA, CS:CODE

START:
    MOV AX, DATA
    MOV DS, AX

    MOV CX, 4
    XOR AX, AX

    LEA SI, ARR
    MOV DI, SI
    ADD DI, 7

REVERSE:
    MOV AL, [SI]
    XCHG AL, [DI]
    XCHG [SI], AL

    INC SI
```

```
25      DEC DI
26
27      LOOP REVERSE
28
29  CODE ENDS
30  END START
```

Output

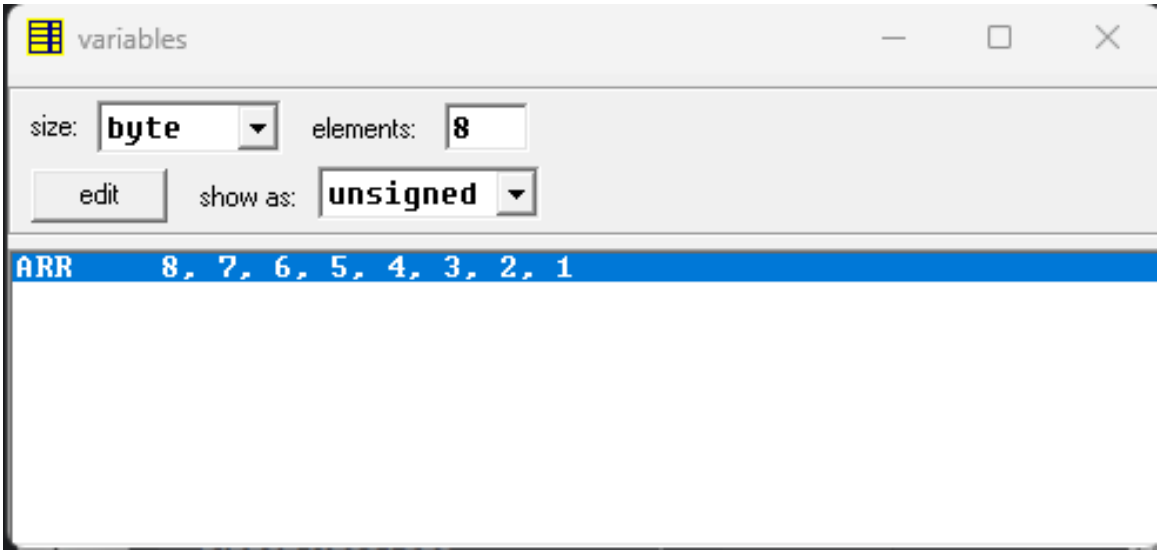


Figure 1: Reversed array.

Problem 7.2

Problem Description

Sort an array.

Process

Code:

```
    —

DATA SEGMENT
    ARR DB 7, 2, 8, 1, 0, 4, 5,
    ↪  9, 6
DATA ENDS

CODE SEGMENT
    ASSUME DS:DATA, CS:CODE

START:
    MOV AX, DATA
    MOV DS, AX

    MOV BX, 8
    XOR AX, AX

    LEA SI, ARR
    MOV DX, SI

SORT_:
    MOV SI, DX
    MOV CX, BX
    MOV DI, SI
    MOV AL, [DI]

BIG_:
    INC SI
    CMP [SI], AL
    JNG NEXT
    MOV DI, SI
    MOV AL, [DI]

NEXT:
    LOOP BIG_

    MOV AL, [SI]
    XCHG AL, [DI]
    MOV [SI], AL

    DEC BX
```

```
39      JNZ SORT_
40
41  CODE ENDS
42  END START
```

Output

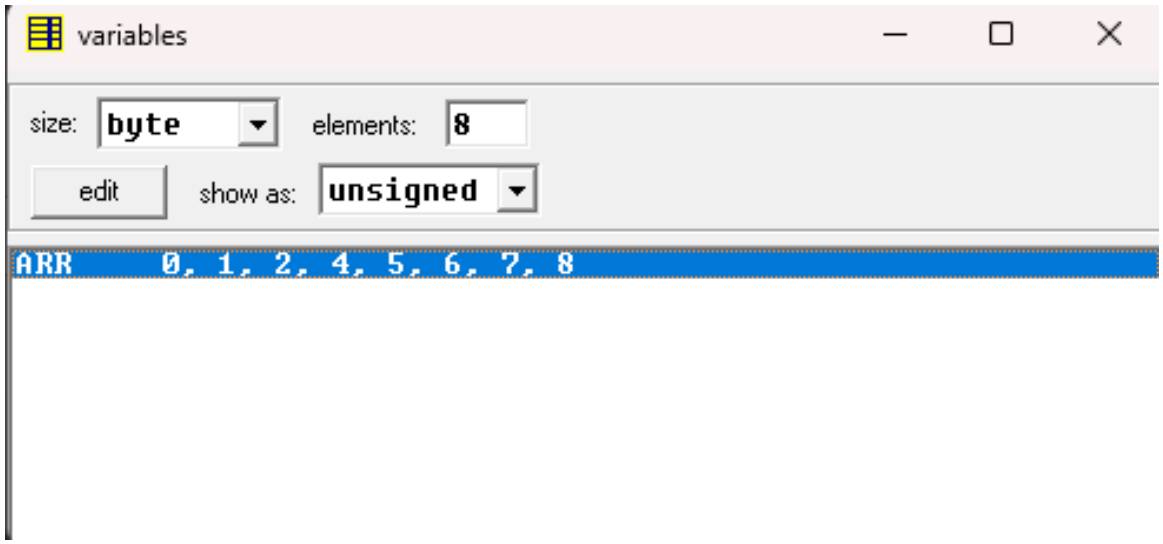


Figure 2: Sorted array.

Problem 7.3

Problem Description

Create and define two segments in memory, then exchange the contents of data segment and code segment.

Process

Code:

```
;--
DATA SEGMENT
    ARR DB 1, 2, 3, 4, 5, 6, 7, 8
DATA ENDS

CODE SEGMENT
    ASSUME DS:DATA, CS:CODE

START:
    MOV AX, DATA
    MOV DS, AX

    MOV AX, 9000H
    MOV ES, AX

    MOV CX, 8
    MOV SI, 0
    MOV DI, SI

SWAP:
    MOV AL, [SI]
    XCHG ES:DI, AL
    XCHG [SI], AL

    INC SI
    INC DI
    LOOP SWAP

CODE ENDS
END START
```

Output

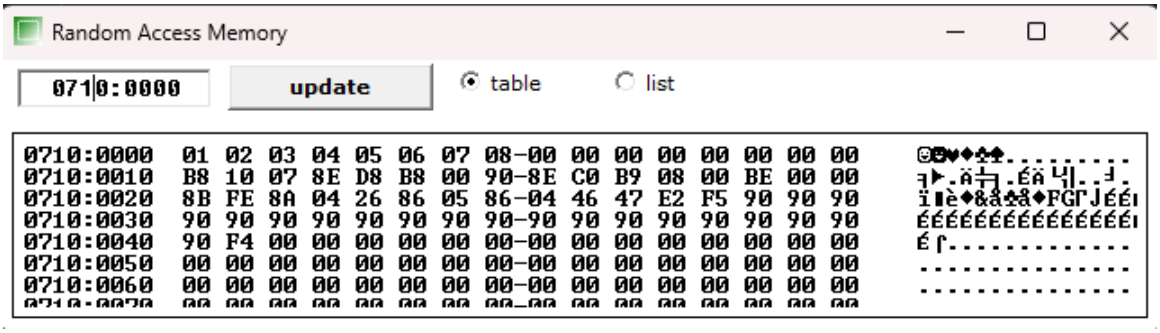


Figure 3: Code segment before swap.

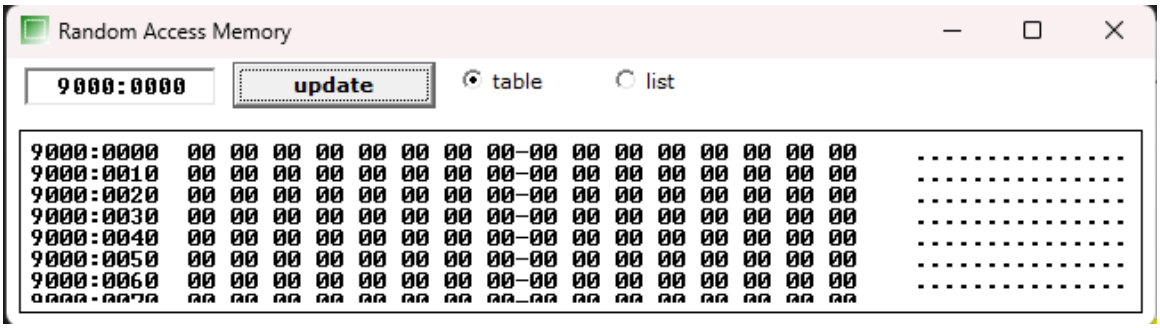


Figure 4: Extra Segment before swap.

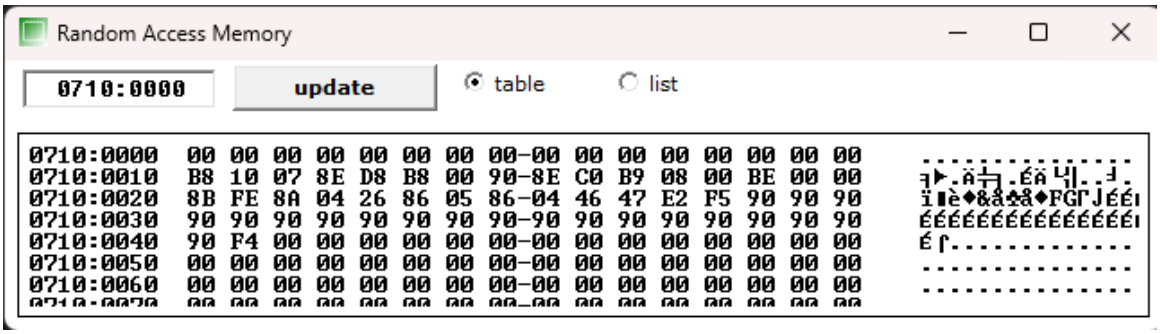


Figure 5: Code segment after swap.

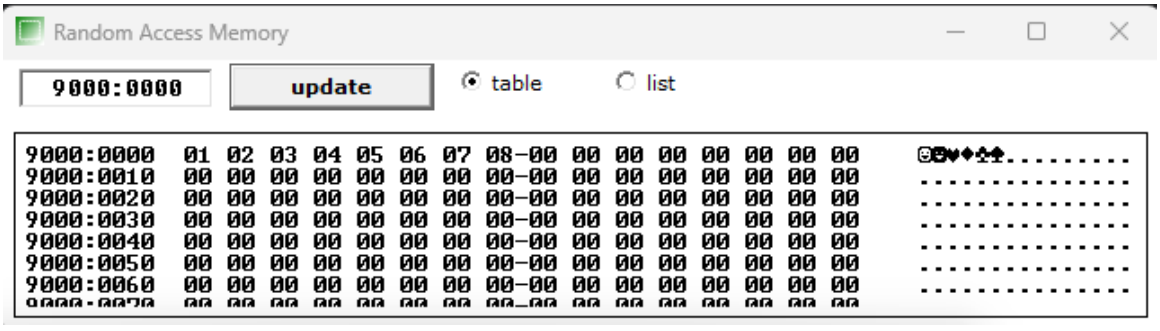


Figure 6: Extra segment after swap.

Keywords

These are the new keywords learned while solving the Problems

ARR DB 1, 2, 3, 4, 5, 6, 7, 8 This is how an array is declared. DB defines the size of each element and putting the values separated by comma defined the elements.